1

$schema "https://opencode.ai/config.json"
permissions
0
action "edit"
resource "*"
effect "deny"
1
action "shell"
resource "*"
effect "ask"
share "disabled"

Resumen de «tienda-libros»
Qué es: una pequeña tienda de libros de ejemplo, desplegada con Docker Compose como 5 servicios contenerizados: un frontend estático servido por Nginx, una API REST, un worker de facturas, PostgreSQL y Redis.

Comportamiento clave
- GET /libros: comprueba Redis (clave "libros"); si no hay caché, consulta Postgres con ORDER BY id y guarda el resultado 60 s (api/src/server.js:14-20).
- POST /pedidos: inserta en la tabla pedidos y hace LPUSH del id en la lista cola:facturas (server.js:22-31).
- Worker: hace BRPOP bloqueante sobre esa lista y registra en consola Generando la factura del pedido <id> (worker.js:8-11). No habla con Postgres ni con la API: la API y el worker solo se comunican a través de la cola de Redis (comunicación asíncrona, desacoplada).
- Frontend: web/index.html es una página estática que hace fetch('/api/libros') y pinta la lista con JS (sin framework ni build).
Red, puertos y volúmenes
- Solo 1 servicio es accesible desde fuera del equipo: el proxy, que publica 8080:80. La API (3000), Postgres (5432) y Redis (6379) no publican puertos: solo existen dentro de la red interna de Compose. El worker tampoco expone nada.
- Dependencias de arranque: proxy → api; api → db (con service_healthy) y cache; worker → cache. La BD tiene healthcheck con pg_isready.
- Volúmenes con persistencia: datos-db (Postgres) y datos-cache (Redis). Montados de solo lectura: nginx/default.conf, web/ y db/init.sql.
Configuración y secretos
- POSTGRES_PASSWORD y JWT_SECRET llegan como variables de entorno desde un .env (solo se versiona la plantilla .env.example con valores de ejemplo); DATABASE_URL y REDIS_URL se construyen en docker-compose.yml.
Cosas que conviene saber (no todo es lo que parece)
- jsonwebtoken está en las dependencias de api/package.json y JWT_SECRET se pasa a la API, pero no se usan en ningún punto del código: no hay autenticación implementada.
- El worker no genera ninguna factura real: solo escribe en el log.
- El proyecto no tiene .gitignore, ni README, ni tests, ni carpeta de documentación. Nada más que los 11 ficheros listados.
- Las rutas de la API no tienen manejo de errores (un await fallido en /libros o /pedidos no está capturado).

El agente no puede cambiar el docker.yml ya que esta en modo plan, ahora solo puede pensar para que en un futuro pueda cambiar los archivos de forma mas eficiente

2

5 servicios en docker-compose.yml, comprobados celda a celda contra los ficheros:

Servicio	Imagen / construcción	Puerto exterior	Puerto interno	Volúmenes	Depende de	Qué hace
proxy	imagen nginx:1.27-alpine	8080 → 80	80	./nginx/default.conf (ro), ./web (ro)	api	Sirve el frontend estático y hace de reverse proxy: /api/ → api:3000
api	build ./api (node:20-alpine)	ninguno	3000	ninguno	db (healthy), cache	API Express: /salud, /libros (con caché 60 s), /pedidos (inserta en BD y encola en Redis)
worker	build ./worker (node:20-alpine)	ninguno	ninguno (no escucha)	ninguno	cache	Consume cola:facturas con BRPOP y «genera» la factura (solo console.log)
db	imagen postgres:16-alpine	ninguno	5432	datos-db → /var/lib/postgresql/data, ./db/init.sql (ro)	ninguno	Base de datos (libros, pedidos) + healthcheck pg_isready
cache	imagen redis:7-alpine	ninguno	6379	datos-cache → /data	ninguno	Doble uso: caché de libros y cola de facturas; persistencia save 60 1

Respuesta corta: 1 de los 5 servicios — solo el proxy.

¿Por qué? Porque en Docker Compose un servicio solo es alcanzable desde fuera del equipo si declara la clave ports (mapeo puerto-host → puerto-contenedor), y en este proyecto el único que lo hace es el proxy:

Servicio	¿Declara ports?	Evidencia
proxy	**Sí:** "8080:80" → accesible desde fuera	docker-compose.yml:4-5
api	No	bloque docker-compose.yml:13-26, sin ports
worker	No	bloque :28-34, sin ports
db	No	bloque :36-50, sin ports
cache	No	bloque :52-57, sin ports

3

Ejercicio 3 — Recursos del proyecto

Clasificación (los 11 ficheros del proyecto + recursos en runtime)

Recurso	Categoría	Dónde vive / quién lo usa	¿Al repo?
web/index.html	Contenido estático	Servido por proxy (nginx) desde el volumen ./web → /usr/share/nginx/html (docker-compose.yml:8)	**Sí**
api/src/server.js	Código de servidor	Copiado a la imagen api (api/Dockerfile:5), se ejecuta en ese contenedor	**Sí**
worker/worker.js	Código de servidor	Copiado a la imagen worker (worker/Dockerfile:5), se ejecuta en ese contenedor	**Sí**
db/init.sql	Datos y esquema	Esquema (libros, pedidos) + 3 libros de ejemplo; lo ejecuta db **solo la primera vez** que se crea el volumen (docker-compose.yml:44)	**Sí** (es esquema, no datos reales)
Volumen datos-db	Datos	Datos reales de PostgreSQL en runtime; gestionado por Docker fuera del proyecto	**No** (no es fichero del repo; si sacas un dump, también fuera)
Volumen datos-cache	Datos	Persistencia de Redis (save 60 1, docker-compose.yml:54), gestionada por Docker	**No**
api/package.json, worker/package.json	Dependencias	Declaran las librerías (express, pg, redis, jsonwebtoken / redis); también definen npm start	**Sí**
node_modules/	Dependencias	**No existe en el proyecto**: se instala dentro de cada imagen con npm install --omit=dev (api/Dockerfile:4)	**No** (ignorar si aparece en local)
Imágenes base	Dependencias	node:20-alpine, nginx:1.27-alpine, postgres:16-alpine, redis:7-alpine; se descargan en build/arranque	No son ficheros (referencias en compose/Dockerfiles)
docker-compose.yml	Configuración	Orquestación de los 5 servicios	**Sí**

docker-compose.yml	Configuración	Orquestación de los 5 servicios	**Sí**
nginx/default.conf	Configuración	Proxy inverso y servicio de estáticos en proxy	**Sí**
api/Dockerfile, worker/Dockerfile	Configuración (build)	Receta de construcción de cada imagen	**Sí**
.env.example	Configuración	Plantilla con valores falsos (cambiame)	**Sí**
.env	**Secretos**	POSTGRES_PASSWORD y JWT_SECRET reales; lo lee docker compose en el host. **Aún no existe**, se crea copiándolo de .env.example	**NO — nunca**
Registros	Registros	No hay ficheros de log: api y worker escriben a stdout (console.log) y nginx/Postgres/Redis también; los captura Docker (docker compose logs)	**No aplica** (si algún día se escriben *.log locales, ignorarlos)

Qué NO debe subirse al repositorio (resumen)

1. **.env** — contiene los dos secretos reales. Es lo único crítico.
2. **Datos de runtime** — volúmenes datos-db/datos-cache y cualquier copia que saques de ellos (.sql.gz, .dump, dump.rdb).
3. **node_modules/** — si alguna vez instalas dependencias en local para depurar.
4. **Registros** — cualquier *.log que se genere fuera de Docker.

Observación extra: JWT_SECRET es secreto por naturaleza aunque el código api/src/server.js aún no lo use (no hay autenticación implementada). Aun así debe tratarse como confidencial.

¿Tiene .gitignore el proyecto? — No

Comprobación (evidencia reproducible):

```
$ ls -la recursos/tienda-libros
drwxrwxrwx  api/     db/     nginx/     web/     worker/
-rw-rw-rw-  docker-compose.yml
-rw-rw-rw-  .env.example
```

No aparece ningún .gitignore (ni en el listado anterior — el único fichero oculto es .env.example — ni dentro de recursos.zip, cuyo índice también he listado). Tampoco hay package-lock.json. Nota adicional: el directorio de trabajo actual **no es todavía un repositorio git**, así que el .gitignore deberá ir en la raíz del repo donde copies el proyecto.

json. Nota adicional: el directorio de trabajo actual **no es todavía un repositorio git**, así que el .gitignore deberá ir en la raíz del repo donde copies el proyecto.

.gitignore propuesto

```
# Secretos: nunca al repositorio (la plantilla sí)
.env
.env.*
!.env.example

# Dependencias instaladas en local (dentro de la imagen se instalan solas)
node_modules/

# Registros
*.log
npm-debug.log*
logs/

# Copias de datos de los volúmenes / backups
*.dump
*.sql.gz
dump.rdb

# Ruido del sistema y editores
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp
```

Por qué cada bloque:

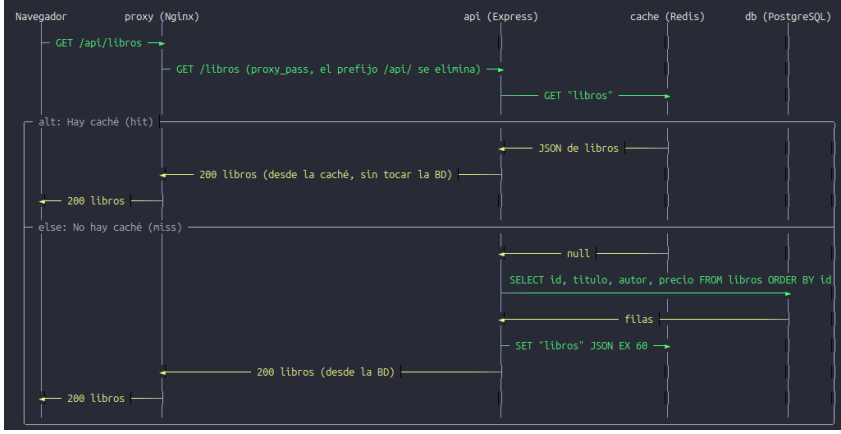
- **.env + .env.* con !.env.example**: ignora cualquier fichero de entorno real, pero conserva la plantilla (la negación es necesaria porque .env.* también casaría con .env.example).
- **node_modules/**: las dependencias no se versionan; se reconstruyen desde los package.json.
- ***.log / logs/**: los logs de Docker no están en el proyecto, pero si alguien redirige salidas a fichero, no deben acabar en el repo.
- ***.dump, *.sql.gz, dump.rdb**: copias de seguridad de los datos. Ojo: **db/init.sql no se ve afectado**, porque el patrón es .sql.gz, no .sql — el esquema sí debe estar en el repo.
- **Ruido de sistema/editores**: no es específico de esta app, pero evita commits sucios.

Jump to latest ↓

4

Dos diagramas de secuencia, verificados contra api/src/server.js, nginx/default.conf y worker/worker.js. Debajo de cada uno tienes la comprobación línea a línea y las correcciones que he aplicado respecto a los típicos errores.

1. GET /api/libros (con caché)



Comprobación contra el código:

Elemento del diagrama	Evidencia
GET /api/libros es la ruta pública, /libros la interna	nginx/default.conf:12-13 (location /api/, proxy_pass http://api:3000/) + api/src/server.js:14
Se consulta primero la caché	server.js:15 (cache.get('libros'))
El hit devuelve lo cacheado sin tocar la BD	server.js:16 (return res.json(...))
El miss lanza el SELECT con ORDER BY id	server.js:17
Tras el SELECT se cachea 60 s	server.js:18 (SET ... EX: 60)
TTL y ausencia de invalidación	No hay ninguna llamada que borre la clave "libros" en todo el proyecto: solo caduca por el TTL

2. POST /api/pedidos (con cola de facturas y worker)



Elemento del diagrama	Evidencia
Validación previa y 400	server.js:23-24 (`if (!libroId
INSERT con RETURNING id	server.js:25-28
LPUSH a la cola **antes** de responder	server.js:29 (antes del 201 de la línea 30)
Respuesta 201 con el id	server.js:30
BRPOP bloqueante (timeout 0)	worker/worker.js:9
El worker solo escribe en el log	worker.js:10 (console.log). No genera PDF, no escribe en la BD, no llama a la API
Comunicación API ↔ worker	Solo vía Redis: LPUSH (server.js:29) / BRPOP (worker.js:9)

Correcciones aplicadas (cosas que un diagrama descuidado suele inventar)

1. **La API no publica /api/libros**: Express sirve /libros; el prefijo /api/ lo añade y lo quita Nginx. El diagrama separa las dos rutas.
2. **El worker no «genera una factura»**: solo hace console.log del id. Sin efectos reales.
3. **El hit de caché no consulta la BD** ni reescribe la caché: retorna directamente.
4. **La cola no se consume antes de responder**: el LPUSH ocurre antes del 201, pero el proceso del worker es asíncrono y sin orden garantizado respecto a la respuesta. El diagrama lo refleja con la nota y sitúa el desbloqueo después del 201.
5. **Nadie invalida la caché** al crear un pedido: solo caduca a los 60 s.

Para verificar que se dibuja en el visor

Estoy en modo Plan, así que no puedo crear el fichero por ti. Cuando pases a Build (o tú mismo):

1. Pega los dos bloques en docs/arquitectura.md dentro del repo del proyecto.
2. En VS Code, abre la vista previa de Markdown (Ctrl+Shift+V) con la extensión «Markdown Preview Mermaid Support» activa.
3. Captura: los dos diagramas renderizados son la evidencia del ejercicio (la captura de mi respuesta sin la vista previa no puntúa).

Si alguno no se dibujara, dímelo con el error exacto del visor y lo ajusto — pero de momento ambos usan solo sintaxis estándar de sequenceDiagram (alt/else/end, Note over, actor/participant), sin caracteres que rompan el parser.

5

1. Qué modelo sigue la aplicación (mi respuesta)

Vista la evidencia, la definición más ajustada es: **una aplicación web distribuida en servicios contenerizados — estilo microservicios en versión mínima (SOA ligera) — desplegada como un único stack Docker Compose sobre un solo host, con flujo de pedidos dirigido por eventos y caché compartida.**

Por qué, con evidencias concretas:

Criterio	Evidencia en el proyecto
Varios servicios, no un monolito	5 contenedores definidos en docker-compose.yml (líneas 2, 13, 28, 36 y 52)
Artefactos propios separados y versionados por su cuenta	api y worker se construyen por separado (docker-compose.yml:14 y :29); versiones independientes: api **1.2.0** (api/package.json:3) y worker **1.0.0** (worker/package.json:3)
Procesos comunicándose por red interna, no por llamadas en memoria	DNS de Compose por nombre de servicio: proxy_pass http://api:3000/ (nginx/default.conf:13), db:5432 (docker-compose.yml:18), cache:6379 (:19 y :31)
Un único servicio habla con la base de datos	Solo api (pool de pg, api/src/server.js:8); el worker no usa Postgres: su única dependencia es redis (worker/package.json:10-11)
Comunicación asíncrona entre servicios	La API y el worker nunca se llaman entre sí: se comunican solo por la lista de Redis cola:facturas (server.js:29 LPUSH / worker.js:9 BRPOP) → API y worker están desacoplados
Patrones de arquitectura distribuida presentes	Reverse proxy / punto de entrada único (nginx), caché-aside con TTL 60 s (server.js:15-18), patrón cola + worker para trabajos de fondo
Estado separado del cómputo	El estado vive en db y cache (con volúmenes); api/worker no guardan datos propios

Dos matices para ser exacto: los servicios propios a medida son **2** (api, worker); proxy, db y cache son imágenes estándar *configuradas*, no construidas. Y todo se despliega **junto** (un compose, un host): imágenes separadas, pero un único despliegue en la práctica (docker compose up).

2. Generalidades de manual que NO se sostienen aquí

Esta es la parte que en una respuesta descuidada (típica de manual) aparecería y que los ficheros desmienten una por una:

Frase típica del «manual»	¿Se sostiene?	Qué la desmiente
«Cada servicio tiene su propia base de datos (datos descentralizados)»	**No**	Hay **una sola** BD PostgreSQL; además solo la usa `api`; `worker` ni siquiera depende de `pg`
«Los servicios se comunican entre sí mediante APIs REST»	**No**	No existe **ninguna** llamada HTTP entre servicios. `api`↔`worker` van por cola Redis; `api`↔`db` por protocolo SQL. El único HTTP es navegador→nginx→api
«Despliegue y escalado independientes de cada microservicio»	**Parcial**	Se pueden reconstruir/reiniciar por separado, pero hay un único `docker-compose.yml`, un solo host, cero réplicas y nginx apunta a una única dirección `api:3000` (`default.conf:13`). Sin orquestador ni pipeline por servicio
«Alta disponibilidad y tolerancia a fallos»	**No**	Un solo host = un solo punto de fallo; sin healthchecks salvo `db` (`docker-compose.yml:45-49`); sin reintentos; la cola no tiene confirmación (BRPOP consume el mensaje)
«nginx actúa como API gateway con seguridad, límites, etc.»	**No**	nginx solo sirve estáticos y hace `proxy_pass` de `/api/`: sin auth, sin rate limiting, sin TLS (`default.conf:1-17`)
«Los servicios se descubren dinámicamente»	**No**	Descubrimiento = DNS interno de Compose por nombre fijo (`api`, `db`, `cache`)

En resumen: es «microservicios» en la **forma** (procesos separados, red, imágenes propias), pero no en el **fondo** (despliegue conjunto, datos centralizados, una sola persona/equipo, sin plataforma). Por eso yo lo llamaría *arquitectura de servicios distribuida de escala mínima*, más que microservicios canónicos.

Checklist para tu contraste con la teoría de la unidad (los criterios clásicos de microservicios, para que puntúes cada uno con tu material):

Checklist para tu contraste con la teoría de la unidad (los criterios clásicos de microservicios, para que puntúes cada uno con tu material):

Criterio clásico	¿Lo cumple?
Servicios pequeños e independientes en ejecución	Sí (5 contenedores, procesos propios)
Base de datos por servicio	**No** (una BD compartida, con un solo cliente)
Interfaces ligeras entre servicios (REST/mensajería)	Solo mensajería (cola Redis); no REST interservicio
Despliegue independiente y automatizado	**No** (un compose, un host, sin CI/CD)
Escalado independiente	En teoría posible, no configurado (sin réplicas)
Observabilidad distribuida / trazabilidad	**No** (cada contenedor loguea por su lado)

3. Ventajas de este diseño para una tienda pequeña

- **El catálogo se lee mucho y se escribe poco:** la caché de 60 s absorbe las lecturas repetidas sin castigar a PostgreSQL (`server.js:15-18`). Ideal para este perfil de tienda.
- **Checkout rápido:** `POST /pedidos` responde 201 en cuanto inserta y encola; la factura no bloquea al cliente (`server.js:29-30`).
- **La cola amortigua caídas del worker:** si el worker está parado, los pedidos se encolan y se procesan cuando vuelva; la tienda sigue vendiendo.
- **Actualizaciones por contenedor:** se puede actualizar la api (1.2.0) sin tocar el worker (1.0.0), porque son imágenes independientes.
- **Superficie de ataque pequeña:** solo el proxy publica puerto (8080); BD y Redis son inalcanzables desde fuera.
- **Portabilidad:** versiones fijadas (`postgres:16-alpine`, etc.) y un `docker compose up` levanta todo igual en cualquier máquina.

4. Inconvenientes de este diseño para una tienda pequeña

- **Complejidad desproporcionada:** 5 contenedores para 3 endpoints; dos almacenes con estado que mantener, respaldar y actualizar (PostgreSQL + persistencia de Redis).
- **Sin alta disponibilidad:** un solo host; si la máquina cae, cae la tienda entera. `restart: unless-stopped` reinicia contenedores, no el servidor.
- **Cola sin garantías:** BRPOP no tiene *ack* (`worker.js:9`): si el worker muere tras extraer un id y antes de terminar, esa factura se pierde sin aviso ni reintento.
- **Coherencia débil:** el INSERT y el LPUSH son dos operaciones no atómicas (`server.js:25-29`); un fallo entre ambas deja un pedido sin tarea de facturación.
- **Sin seguridad de producción:** HTTP plano y sin autenticación (el `JWT_SECRET` anda suelto sin usarse); habría que añadir TLS y control de acceso en el proxy o la API.
- **«Microservicios» sin los beneficios:** no hay despliegue por pipelines, ni trazabilidad, ni escalado real; se paga el coste de lo distribuido sin cobrar sus ventajas. Para una sola persona manteniendo una tienda, un monolito modular sería más barato de operar.
- **Caché sin invalidación:** hasta 60 s de datos viejos tras cualquier cambio en `libros` (aceptable en catálogo, no lo sería para stock).

6

6. Ficha de requerimientos de implantación — tienda-libros

Leyenda: ✅ = ya lo trae la aplicación · **+** = hay que añadirlo para producción
Supuestos: tienda pequeña, tráfico bajo (decenas de pedidos/día), un servidor, mantenida por una persona.

Área	✅ Ya lo trae	+ Añadir para producción
Instancia	Contenedores ligeros (alpine) y portables	Servidor dimensionado, SO LTS endurecido
Software	Versiones fijadas por rama de imagen	Lockfiles npm, Node 20 (EOL) → 22/24, registry, CI
Red	Solo el proxy expuesto (8080); red interna aislada	Dominio + DNS, TLS, 80/443, cortafuegos
Seguridad	Secretos fuera del repositorio (`.env.example`)	HTTPS, autenticación, gestión/rotación de secretos, no-root
Entornos	Un compose funcional	dev/staging/prod, CI/CD, estrategia de migraciones
Datos	Volúmenes persistentes; persistencia de Redis; esquema en `init.sql`	Copias automáticas + offsite + restauración probada
Monitorización	Endpoint `/salud`; logs a stdout	Healthchecks completos, rotación de logs, alertas
Despliegue y vuelta atrás	`docker compose up -d`; `restart: unless-stopped`	Imágenes etiquetadas, procedimiento y simulacro de rollback

6.1 Hardware o instancia

- ✅ La app no exige hardware especial: 5 contenedores alpine + 2 builds de Node; en marcha consume del orden de 500 MB de RAM. Estado únicamente en los volúmenes `datos-db` y `datos-cache`.
- **+ Servidor:** 1 VM tipo VPS con **2 vCPU / 4 GB RAM / 40 GB SSD** (sobra para esta escala; ~10–25 €/mes en proveedores tipo Hetzner, DigitalOcean u OVH). Alternativa on-premise: mini-PC equivalente.
- **+ SO:** Debian 13 o Ubuntu 24.04/26.04 LTS, actualizado; usuario de despliegue no-root con sudo; acceso SSH solo con clave.
- **+ Riesgo asumido:** un solo host = un punto único de fallo; aceptable para tienda pequeña pero debe quedar documentado. Vía de crecimiento: escalar verticalmente → mover BD/Redis a servicios gestionados (solo hay que cambiar `DATABASE_URL`/`REDIS_URL`) → multi-host (rediseño).
- **+ Mejor construir las imágenes en CI** y desplegar con `pull` (ahorra RAM/CPU al servidor durante los builds).

6.2 Software y versiones

Componente	✅ Lo que trae hoy	+ Para producción
Proxy	`nginx:1.27-alpine` (2024)	Fijar una rama estable actual
Base de datos	`postgres:16-alpine` (soportada hasta 9-nov-2028 ✅)	Mantener; actualizar de versión menor con `pg_upgrade` cuando proceda
Caché/cola	`redis:7-alpine`	Vigilar rama soportada (valorar Valkey/Redis 8 por licencia)
Node (build de api/worker)	`node:20-alpine` — **Node 20 alcanzó EOL el 30-abr-2026** ⚠	Migrar los Dockerfiles a `node:22` o `node:24` LTS
Dependencias npm	`express ^4.19.2`, `pg ^8.11.5`, `redis ^4.6.13`, `jsonwebtoken ^9.0.2` (sin usar)	**Generar y versionar** `package-lock.json` **y usar** `npm ci --omit=dev` (hoy los builds no son reproducibles: `npm install` con rangos `^`)
Host	—	Docker Engine + plugin Compose v2 del repositorio oficial de Docker
Artefactos	—	Registry (p. ej. ghcr.io) con imágenes etiquetadas por versión
Mantenimiento	—	`unattended-upgrades` para el SO; escaneo de imágenes (trivy/grype) y `npm audit` periódicos

Referencias: calendario de Node.js (https://nodejs.org/en/about/previous-releases) y ciclo de vida de PostgreSQL (https://endoflife.date/postgresql).

6.3 Red (dominio, DNS, puertos, cortafuegos)
- ✅ Topología ya adecuada: un único puerto publicado hacia fuera (8080→80); API, BD y Redis solo en la red interna de Compose, accesibles por nombre (api, db, cache); nginx sirve la web y enruta /api/.
- ➕ Dominio y DNS: un dominio (p. ej. tienda.example.org) con registro A → IP del servidor; TTL bajo (300 s) mientras se ajusta. En producción, sustituir server_name _ por el dominio real.
- ➕ TLS: certificado Let's Encrypt (certbot con webroot o renovación automática por timer) y, en nginx, 443 ssl + redirección 80→443. Alternativa: Caddy/Traefik delante para terminar TLS. Certificados montados en solo-lectura.
- ➕ Publicación de puertos: cambiar el mapeo a 80:80 y 443:443 (o mantener 8080 solo si hay otro proxy delante). El resto de puertos (3000, 5432, 6379) permanece sin publicar, como ya está.
- ➕ Cortafuegos: permitir solo 22 (restringido a IPs de administración), 80 y 443. Usar el cortafuegos del proveedor si existe (grupo de seguridad). Ojo con el detalle clásico: Docker publica saltándose las reglas de ufw, así que o se filtra en el grupo de seguridad externo o se añaden reglas en la cadena DOCKER-USER; no basta con ufw deny a secas.
- ➕ Salida a Internet para el servidor: renovación de certificados y despliegues (restringible a registry + ACME).
6.4 Seguridad y secretos
- ✅ Los secretos no están en el repositorio (solo la plantilla .env.example); llegan a los contenedores como variables de entorno (POSTGRES_PASSWORD, JWT_SECRET); la BD y la caché son inalcanzables desde fuera; imágenes alpine de superficie reducida.
- ➕ Prioridad 1 — HTTPS: hoy todo va en HTTP plano. Es el primer requisito.
- ➕ Prioridad 2 — autenticación: la aplicación no trae ninguna: POST /pedidos está abierto a cualquiera; jsonwebtoken está declarado y JWT_SECRET se inyecta, pero no se usan (api/src/server.js no importa la librería). Para una tienda real: implementar auth (el JWT ya está previsto) o limitar el acceso. En la ficha no se escriben valores de secretos, solo nombres.
- ➕ Gestión de secretos: .env con permisos 600 y propietario del usuario de despliegue; mejor aún, Docker secrets o un gestor (Vault/SOPS). Rotación periódica de POSTGRES_PASSWORD y JWT_SECRET.
- ➕ Endurecer contenedores: usuario no-root en ambos Dockerfiles, read_only donde sea posible, límites de mem_limit/cpus, no montar nunca el socket de Docker.
- ➕ Aplicación: rate limiting en nginx (evitar inundar la cola de facturas), cabeceras de seguridad (X-Content-Type-Options, CSP para la estática...), eliminar jsonwebtoken si no se va a implementar auth.
- ➕ Host: SSH sin root y sin contraseña, fail2ban, actualizaciones automáticas de seguridad, y copias cifradas.
6.5 Entornos
- ✅ Un docker-compose.yml que levanta todo; NODE_ENV: production fijado en el compose; configuración por variables de entorno.
- ➕ Tres entornos: desarrollo (compose en la máquina local), staging (VM pequeña o el mismo servidor con otro nombre de proyecto y puertos; imprescindible para ensayar despliegues y rollback) y producción.
- ➕ Un .env por entorno, jamás en git; valores distintos (secretos y contraseñas propias en cada uno).
- ➕ CI/CD (p. ej. GitHub Actions): build → escaneo → push a registry → despliegue por SSH (docker compose pull && docker compose up -d). Repositorio etiquetado por releases (git tags).
- ➕ Migraciones de esquema: hoy db/init.sql solo se ejecuta la primera vez que se crea el volumen; cualquier cambio posterior en tablas no se aplica solo. Adoptar una herramienta de migraciones (node-pg-migrate, Flyway...) o un procedimiento manual documentado con dump previo.
6.6 Datos y copias de seguridad
- ✅ Volumen datos-db para PostgreSQL (sobrevive a recrear contenedores), volumen datos-cache con persistencia RDB (save 60 1), y esquema + semilla recreables desde db/init.sql.

- ➕ Hoy no hay ninguna copia configurada: es el mayor riesgo en producción.
- ➕ PostgreSQL: pg_dump -Fc programado (cron/systemd timer, p. ej. diario a las 03:00) → cifrado → copia fuera del servidor (S3/Backblaze/otro host) con retención, p. ej. 7 diarias / 4 semanales / 12 mensuales. Probar la restauración en staging una vez al trimestre (una copia sin restauración probada no es una copia). Dump manual antes de cada despliegue con cambios de datos o esquema.
- ➕ Redis: contiene la caché (reconstruible) y la cola de facturas. Documentar la decisión: aceptar su pérdida (los pedidos en cola se perderían; BRPOP ya es destructivo) o subir a AOF y copiarla. Para esta escala, lo razonable es aceptarlo y documentarlo.
- ➕ Objetivos: RPO ≤ 24 h (mejorable con dumps horarios), RTO razonable de 1–2 h (provisionar VM + desplegar + restaurar dump). Dejar escrito el procedimiento de restauración paso a paso y quién lo ejecuta.

6.7 Monitorización y registros

- ✅ Endpoint GET /api/salud (→ /api/salud a través del proxy); todos los servicios escriben a stdout; healthcheck de PostgreSQL usado para ordenar el arranque.
- ➕ Healthchecks en Compose para todos: api (wget -qO- localhost:3000/salud), cache (redis-cli ping), proxy (HTTP a sí mismo), worker (latido de proceso). Usarlos también en depends_on.
- ➕ Rotación de logs: el driver json-file de Docker no rota por defecto y puede llenar el disco; configurar max-size: 10m, max-file: 3–5 (o centralizar en Loki/Promtail, o journald).
- ➕ Monitorización externa: ping a https://dominio/api/salud y a la web cada minuto (Uptime Kuma autoalojado o servicio equivalente), con aviso por email/Telegram.
- ➕ Métricas: Netdata (simple) o node_exporter + cAdvisor + Prometheus/Grafana: CPU, RAM, disco, reinicios de contenedores.
- ➕ Alertas concretas: disco > 80 %, contenedor caído/reiniciando en bucle, longitud de la cola LLEN cola:facturas creciendo (señal de que el worker está caído y las facturas no se generan), errores 5xx en nginx, certificado TLS a menos de 15 días.
- ➕ Retención de logs acordada (p. ej. 14 días) y, si se añaden datos personales de clientes, revisar RGPD.

6.8 Fases del despliegue

- ✅ docker compose up -d --build construye y arranca en orden (BD healthy → api; cache → worker); restart: unless-stopped recupera caídas.
- ➕ Fase 0 — Preparación (antes del día D): VM creada y endurecida (6.1, 6.4); DNS apuntando; imágenes construidas y etiquetadas en el registry; secretos generados y .env creado con permisos 600; copia de seguridad inicial si se migran datos existentes.
- ➕ Fase 1 — Despliegue inicial: clonar el repo en el servidor (/opt/tienda); docker compose pull; docker compose up -d; esperar a que todo esté healthy.
- ➕ Fase 2 — Verificación funcional (checklist con evidencia): curl https://dominio/api/salud → {"estado":"ok"}; GET /api/libros devuelve el catálogo; la web carga; un pedido de prueba responde 201 y el worker registra la factura en sus logs; borrar el pedido de prueba.
- ➕ Fase 3 — TLS y DNS: emitir certificado, activar 443 + redirección, probar la renovación (certbot renew --dry-run).
- ➕ Fase 4 — Copias y monitorización activadas (6.6 y 6.7) antes de anunciar el servicio.
- ➕ Fase 5 — Anuncio y vigilancia reforzada 24–48 h.
- ➕ Actualizaciones rutinarias: dump previo si hay cambios de datos → pull de la nueva imagen → up -d del servicio → verificación → conservar la imagen anterior para poder volver atrás. Aviso: init.sql solo corre con volumen vacío; una migración de datos se hace restaurando dump, no confiando en la semilla.

6.9 Plan de vuelta atrás

- ✅ restart: unless-stopped recupera caídas (no es un rollback); el repositorio git permite reconstruir la versión anterior del código; los volúmenes conservan los datos.

- ➕ Preparación previa: etiquetar cada release (git tag + imágenes api:v1.2.0, worker:v1.0.0 en el registry). Hoy las imágenes se construyen en el servidor sin etiqueta propia: sin registry+tags el rollback depende de reconstruir código viejo (lento e inseguro).
- ➕ Disparadores para volver atrás: healthchecks en rojo, errores 5xx sostenidos, pedidos que no se registran, worker que no vacía la cola, degradación grave. Decisor: la persona responsable; tiempo máximo de diagnóstico antes de decidir (p. ej. 30 min).
- ➕ Rollback de código (minutos): parar el servicio afectado → desplegar la versión etiquetada anterior → verificar salud y función → conservar logs e imagen defectuosa para el análisis.
- ➕ Rollback de datos (último recurso): restaurar el último dump validado; se pierden los pedidos posteriores (es el RPO asumido). Al no haber migraciones automáticas, política: cambios de esquema compatibles hacia atrás + restore como red de seguridad. Avisar y registrar.
- ➕ Simulacro obligatorio: desplegar y volver atrás una versión de prueba en staging antes de la apertura y tras cambios mayores. Documentar el resultado.
- ➕ Post-incidente: análisis de causa y actualización de esta ficha.

6.10 Checklist «antes de abrir» (bloqueantes)

1. HTTPS con renovación automática probada.
2. Copias diarias offsite + restauración probada en staging.
3. Secretos fuertes, .env con permisos 600 (o gestor), fuera de git.
4. Node 20 → 22/24 y package-lock.json + npm ci.
5. Cortafuegos (solo 22/80/443) y SSH endurecido.
6. Healthchecks + alertas (disco, contenedores, cola, TLS).
7. Imágenes etiquetadas en registry + simulacro de rollback.
8. Decisión sobre autenticación de POST /pedidos (implementar o restringir).